

CT2Yarn: Yarn-Level Reconstruction of Crochet from Computed Tomography

Chang Luo¹ and Nobuyuki Umetani¹

The University of Tokyo, Japan

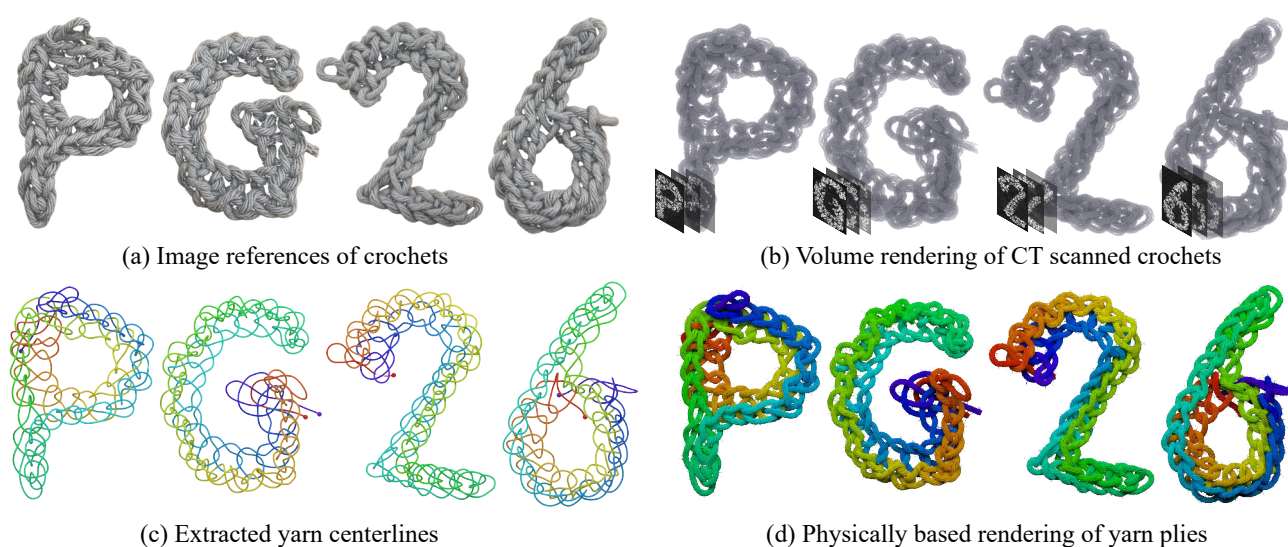


Figure 1: From real crocheted samples to a continuous yarn path. Starting from real crocheted samples (a), we acquire micro-CT scans that reveal their internal structure (b). Our pipeline then reconstructs a single continuous yarn centerline (c), visualized with colors indicating arc length, and generates a ply-level yarn geometry for rendering (d).

Abstract

We introduce CT2Yarn, a human-in-the-loop framework for recovering a single continuous yarn path from micro-computed tomography (micro-CT) scans of real crochet objects. Crochet is a craft that creates complex three-dimensional shapes by interlocking loops formed from a single yarn. Recovering the underlying yarn path from external observations is challenging because of severe self-occlusion. While micro-CT reveals the full internal structure of a crochet object, the volumetric scan alone does not explicitly encode how the yarn traverses the object. This challenge stems from the hierarchical structure of yarn: a yarn consists of multiple twisted plies, and each ply itself consists of twisted fibers. Consequently, local fiber orientations observed in micro-CT scans are not aligned with the overall yarn direction. To recover yarn-level orientations from ply-level fiber orientations, we first estimate local fiber directions using Gabor filtering and convert the volume into an oriented point cloud. We then introduce an anisotropic mean-shift procedure that aggregates local fiber orientations within a neighborhood of the yarn radius into yarn-level orientation estimates. Combined with an automatic topology skeletonization strategy, our method extracts yarn-path fragments. Subsequent fragment linking, junction cleaning, and loop detection merge these trees into a small number of long curves. Where the automatic reconstruction remains ambiguous, a sketch-based user interface enables users to interactively complete the single continuous yarn path. The recovered yarn path enables downstream applications including physically based simulation, ply-level rendering, and stitch-pattern extraction. Our source code is publicly available at <https://github.com/netbeifeng/ct2yarn>.

CCS Concepts

• **Computing methodologies** → **Parametric curve and surface models**; **Volumetric models**; **Shape analysis**;

1. Introduction

Crochet is a textile craft that creates complex three-dimensional structures by interlocking loops formed from a single continuous yarn. Owing to its flexibility and accessibility, crochet is widely used to produce garments, plush toys, and artistic works. In computer graphics, various studies have explored the physically based simulation [KJM08; YKJM12] and photorealistic rendering [ZJA*23; HSW*25] using explicit yarn-path representations. However, almost all computational work on yarn modeling has focused on synthetic structures [WSY19; GLNM20; EPIB22], where the yarn path is procedurally generated from an idealized stitch pattern. Recovering the yarn path of real crocheted objects is essential for faithfully digitizing handmade crochet and enabling downstream applications such as simulation and rendering, yet remains largely unexplored.

A natural starting point for recovering yarn structure from real samples is image-based reconstruction. Recent work has made progress on woven fabrics [JWH*22; LSS*26] and on knit fabrics [KOM*19], but all assume the fabric forms a regular repeating grid. However, unlike knitting, where stitches are connected in a regular row-by-row manner, crochet stitches can be worked into arbitrary earlier stitches, producing freeform topologies that cannot be naturally represented by a regular grid. Moreover, interlocked loops occlude a substantial portion of the yarn from every viewpoint. Consistent with these challenges, existing crochet-specific tracking systems rely on hand-worn motion sensors rather than visual inputs [MLB25], and recent benchmarking efforts indicate that vision-language models fail to reliably recover abstract crochet patterns from images [LHHC25].

Micro-computed tomography (micro-CT) avoids the limitation of occlusion by directly imaging the interior of a physical sample using X-rays. Previous work has used this volumetric imaging modality to reconstruct woven fabric [ZJMB11; ZLB16] and hair [SSW*23]. Although modern CT scanners provide voxel resolutions well below typical yarn diameters, recovering yarn structure from the resulting density volumes remains challenging. Unlike hair, whose strands are separated in air with clear boundaries, neighboring yarn segments in crochet are brought into tight contact at every stitch, causing their boundaries to become ambiguous (Fig. 3). Furthermore, the fiber orientations within a yarn are not aligned with the yarn direction itself. A yarn consists of multiple twisted plies, each of which is in turn composed of twisted fibers. Consequently, the local fiber orientations observed in the CT volume provide only indirect information about the underlying yarn trajectory. Together, these challenges make individual yarn paths difficult to distinguish in the volume. Furthermore, crocheted objects exhibit a strong topological constraint: they are constructed from a single continuous yarn. Unlike hair reconstruction, where strands are largely independent, an erroneous connection in crochet can propagate globally and corrupt the recovered yarn path.

We present *CT2Yarn*, a human-in-the-loop framework that combines an automatic reconstruction pipeline with lightweight interactive editing to recover a single continuous yarn path from the micro-CT density volume of a crocheted object. Given a 3D density volume as input, our method reconstructs a centerline curve representing the entire yarn. To account for the hierarchical struc-

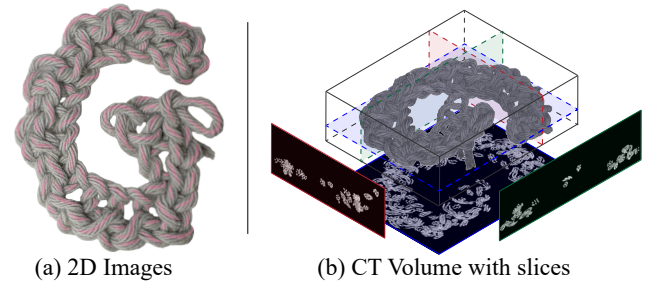


Figure 2: Photograph and micro-CT volume of the same crocheted sample. (a) Interlocking loops occlude most of the interior yarn from any single viewpoint. (b) The CT volume, rendered with three orthogonal slices, exposes every yarn cross-section throughout the fabric.

ture of yarn, which consists of multiple plies, our yarn orientation estimation proceeds in two stages. We first estimate a voxel-wise orientation field of local plies using Gabor filtering. We then infer yarn-level orientations from the estimated ply orientations using an anisotropic mean-shift clustering procedure. The yarn centerline candidates are extracted by connecting neighboring mean-shift points into a graph. However, the recovered centerline candidates are fragmented due to the tight contacts between neighboring yarn segments. We therefore resolve this by a heuristic linking strategy and perform cleaning as the automatic pipeline. The automatic result provides a good starting point, and an interactive interface lets users resolve the remaining ambiguous connections, completing the final single continuous yarn path.

We evaluate our method on micro-CT scans of physical crocheted samples covering various stitch types. We evaluate primarily through visual comparison, complemented by a quantitative measures, ablation studies of the main algorithmic components, and a demonstration of stitch-pattern extraction via template fitting. As no ground-truth yarn geometry is available for real crocheted objects, the quantitative measures are computed against manually annotated references, and a synthetic CT volume with an exact ground truth. We also quantify the manual effort of the interactive completion, at a median of four sketch edits and under three minutes per sample. The resulting yarn representation supports downstream applications including physically based simulation, ply-level rendering, and stitch-pattern extraction. To support future research, we release both the micro-CT volumes and the reconstructed yarn centerline curves. Our contributions are as follows:

- A two-stage orientation analysis that infers yarn-level trajectories from ply-level fiber orientations observed in micro-CT volumes.
- An automatic repair stage that merges fragmented centerline candidates into a small number of long curves, and a sketch-based interface for the few connections it cannot resolve.
- A publicly available dataset consisting of micro-CT scans and reconstructed yarn curves of real crocheted samples.

2. Related Work

2.1. Yarn-Level Modeling of Knitted Fabrics

The regular row-and-column structure of knit stitches has enabled an extensive yarn-level pipeline in computer graphics. The stitch mesh [YKJM12] represents knit fabric as a quad-dominant mesh whose faces correspond to stitches, and has been extended to automatic generation from arbitrary 3D shapes [WGF*18] and to knittability-preserving editing [WSY19]. Industrial knitting machines are themselves driven by formal compilers and visual programming interfaces [MAN*16; NWYM19], enabling automated fabrication of complex 3D knit garments. Solid knitting extends this automated fabrication from hollow garments to dense volumetric objects, and the interactive design tool skCAD [HLI26] lets users design such solids on a three-dimensional stitch lattice with automatically generated machine-executable patterns.

Yarn-level simulation, initiated by Kaldor et al. [KJM08], has been refined through persistent contact handling [CLO17], accelerated via homogenization [SNW20; YSL*24], and embedded in interactive tools for authoring and relaxing knit and woven patterns on a periodic grid [LWS*18]. Alongside simulation, fiber- and ply-level rendering models reproduce knit and woven appearance at varying scales [WY19; ZJA*23; HSW*25], with recent neural approaches further accelerating woven fabric rendering [CWW24].

This mature pipeline is built around a single structural assumption: that the fabric is organized as a regular grid. It lets every stitch be addressed by row-and-column indices, a rule shared by all of the pipelines above. Crochet violates this assumption: its stitches can be worked into any earlier stitch, producing freeform topologies that no strict row-and-column grid captures.

2.2. Computational Approaches to Crochet

Computational work on crochet is more recent and smaller in scope, focusing on forward synthesis, pattern representations, machine fabrication, and process tracking. Early efforts convert target geometric forms into crocheting recipes [ÇBG17], adapt the knit stitch mesh representation to produce yarn-level geometry [GLNM20], and map a 3D character mesh to a crochetable pattern for amigurumi [EP1B22], following the interactive plush-design tool Knitty [IIS08]. Storck et al. [SGSK22] model each stitch as a unit cell of parameterized key points along the yarn centerline, interpolated by Kochanek-Bartels splines and exported to finite element simulation. Beyond synthesis, Seitz et al. [SRLH22] introduce a visual domain-specific language for patterns, [SFK23; SFFK23] develop a design tool for the crochet machine prototype, StitchFlow [MLB25] infers the crochet progress from inertial measurements of the crafter's hand.

None of these works recovers the 3D yarn structure of a physically aligned crocheted sample. The closest in output, Storck et al.'s topology model [SGSK22], also produces a continuous yarn curve, but synthesizes it from a fixed per-stitch key-point template with tension as a parametric scaling, remaining as a synthetic model rather than a reconstruction of a real crocheted object.

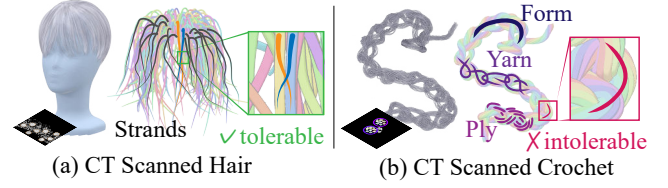


Figure 3: Hair vs. crochet: hierarchy and error tolerance. (a) Hair has independent strands and tolerates locally-plausible errors. (b) Crochet has three levels (form, yarn, plies). Because the entire object is constructed from a single continuous yarn, a local reconstruction error will propagate globally.

2.3. Image-Based Yarn and Strand Reconstruction

Image-based methods recover yarn or strand structure directly from photographs or microscopy of real samples. For woven fabrics, an extensive line of inverse-rendering work recovers procedural yarn parameters together with appearance properties from photographs [SZK15; KSZ*16; GHC17; JWH*22; TLH*24; WKZ*25], and a recent extension reaches fiber-level detail from a single microscopic image [LSS*26]. For knit fabrics, image-to-instruction pipelines infer the discrete stitch label at each cell of an assumed grid [KOM*19; TMK*19]. For hair, strand-accurate multi-view capture reconstructs many independent 3D strands from a calibrated multi-view-stereo setup [NWKS19]. Fabric pipelines require a regular grid that fixes where each yarn or stitch can lie, whereas strand pipelines assume independent, externally visible filaments emerging from a known support surface. Neither holds for crochet, whose interlocking loops occlude most of the interior yarn from any single viewpoint (Fig. 2(a)).

2.4. CT-Based Yarn and Strand Reconstruction

In textile engineering, Gong et al. [GOS09] modeled yarn cross-sections in plain woven fabric from micro-CT scans. Shinohara et al. [STOK10] pioneered CT yarn centerline extraction on regular knit and woven fabrics. Later work probes single-yarn microstructure [HLN*24], manufacturing differences [JZY*26], and woven appearance modeling in graphics [ZJMB11; KSZ*16]. These works assume regular grids or isolated straight yarns, enabling per-yarn tracing without contact disambiguation. Closer to our setting, Zhao et al. [ZLB16] fit a procedural yarn model to CT scans of yarn samples, recovering statistical parameters of fiber arrangement within a ply. Their input is short parallel yarn segments, and their output describes a yarn material rather than the path of any specific yarn through a fabric. On the other hand, CT2Hair [SSW*23] reconstructs independent hair strands from a CT volume. Hair strands are rooted on a scalp and remain largely separated, so reconstruction reduces to tracing individual strands from their roots. A crocheted object instead consists of a single continuous yarn of twisted plies whose loops repeatedly come into close contact, so local fiber orientations do not reveal the yarn trajectory and neighboring passes are hard to distinguish in the volume. The recovered centerline must moreover form a single continuous yarn path consistent with the physical crochet structure, a domain gap that prevents hair-reconstruction methods from being applied directly.

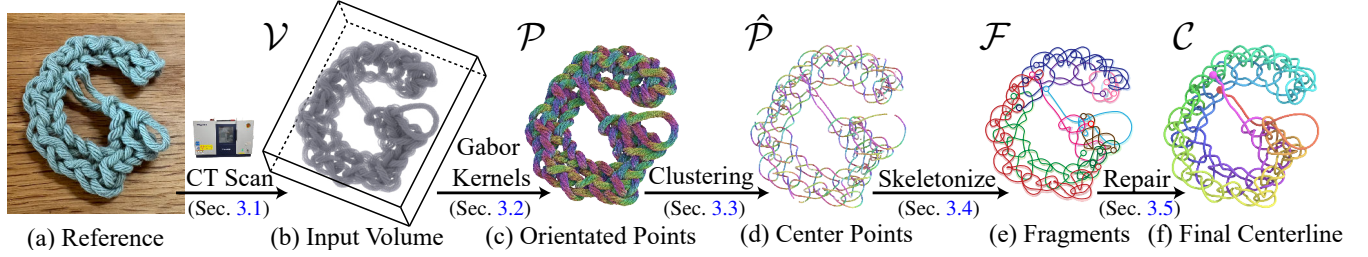


Figure 4: Pipeline overview. (a) Reference image of the physical crocheted sample. (b) Micro-CT volume $\mathcal{V}$. (c) Per-voxel orientation field from 3D Gabor responses, shown as the oriented point cloud $\mathcal{P}$. (d) Center points $\hat{\mathcal{P}}$ after anisotropic mean-shift. (e) Yarn fragments $\mathcal{F}$ extracted from the center points. (f) Final single continuous centerline $\mathcal{C}$ after automatic repairing and interactive completion.

3. Method

Given a density volume $\mathcal{V} \in \mathbb{R}^{H \times W \times L}$ from a micro-CT scan, our goal is to recover a single continuous yarn path $\mathcal{C} = (\mathbf{p}_1, \dots, \mathbf{p}_N)$, an ordered sequence of center points $\mathbf{p}_i \in \mathbb{R}^3$ whose ordering encodes the topology of the entire yarn. Our pipeline consists of three stages (Fig. 4). *Orientation estimation* (Sec. 3.2) assigns a fiber direction vector to each voxel above a density threshold, resulting in a sparse oriented point cloud $\mathcal{P}$. *Center-point extraction* (Sec. 3.3) applies anisotropic mean-shift clustering to $\mathcal{P}$ to recover yarn centerline candidates $\hat{\mathcal{P}}$. *Topology skeletonization* (Sec. 3.4) groups $\hat{\mathcal{P}}$ into ordered yarn fragments $\mathcal{F}$ and assembles them into the final single continuous yarn path $\mathcal{C}$. We additionally provide an interactive sketch interface for manually resolving ambiguous fragment connections when the automatic reconstruction is insufficient (Sec. 3.5). The recovered yarn path $\mathcal{C}$ is then used in downstream applications including simulation, rendering, and stitch-pattern extraction.

3.1. Preprocessing

A micro-CT scanner reconstructs a density volume $\mathcal{V}$ from X-ray projections taken around the sample [FDK84]. Because yarn is a soft fibrous material with weak X-ray attenuation, its boundary with air is not sharply defined. Consequently, the reconstructed density field exhibits substantial blur and noise, together with a low-intensity halo surrounding the yarn, as also observed by CT2Hair [SSW*23]. To suppress this haze, we threshold the non-zero voxels with Otsu’s criterion [Ots79], which picks one threshold τ per volume automatically. We avoid additional smoothing in order to preserve fine boundary structures, and use the thresholded volume in all subsequent stages.

3.2. Orientation Field Estimation

This stage assigns a tangent direction to every voxel inside the thresholded volume and extracts a sparse, oriented point cloud over regions where the local structure is reliably line-like. Hair-CT pipelines such as CT2Hair [SSW*23] estimate orientations from voxel intensity gradients. This fits hair’s structure as presented in Fig. 3a: individual strands sit in air with clear boundaries, so each voxel’s gradient encodes the local strand direction. Crochet has no such clean boundaries, so gradients are unreliable. We instead estimate orientation with a Gabor structure-tensor formulation [KWA11]: each 3D Gabor kernel is bandpass at the yarn scale,

so it responds to the whole yarn-scale ridge rather than to a boundary. We sample K carrier directions $\mathbf{d}_k$ uniformly on the upper hemisphere via a Fibonacci lattice (Fig. 5b). For each direction, we construct a 3D Gabor kernel whose width is determined by the yarn radius R (measured from a CT slice) and whose carrier wave is aligned with $\mathbf{d}_k$. We evaluate the convolutions efficiently in the frequency domain using FFTs. The squared response $|r_k(\mathbf{x})|^2$ is large when the local fiber orientation aligns with $\mathbf{d}_k$ at $\mathbf{x}$. Summing over all directions gives the per-voxel structure tensor

$$\mathbf{T}(\mathbf{x}) = \sum_{k=1}^K |r_k(\mathbf{x})|^2 \mathbf{d}_k \mathbf{d}_k^\top, \quad (1)$$

The eigendecomposition of $\mathbf{T}$ then reveals the dominant local fiber orientation. In a locally linear neighborhood, one eigenvalue dominates and the smallest eigenvector aligns with the principal fiber direction. We take this smallest eigenvector as the per-voxel local tangent direction $\mathbf{t}(\mathbf{x}) \in \mathbb{R}^3$, where $|\mathbf{t}(\mathbf{x})| = 1$.

We then keep only voxels with a strong enough Gabor response. The mean per-direction response $\bar{r}(\mathbf{x}) = \frac{1}{K} \sum_k |r_k(\mathbf{x})|$ is large inside fiber material and small in air, background, or the surrounding haze, so we drop voxels whose mean response falls below τ_E . Averaging over the K directions makes τ_E independent of the number of Gabor directions. Finally, we reduce the density of the remaining samples by voxel binning, keeping one point per cubic cell of side length d times the mean voxel spacing. The resulting sparse oriented point cloud is denoted by $\mathcal{P} = \{(\mathbf{p}_i, \mathbf{t}_i)\}$.

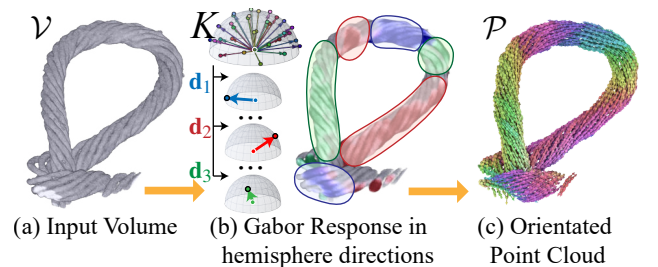


Figure 5: Orientation field estimation. (a) Input CT volume. (b) Regions running in different directions respond to their respective Gabor kernels, shown here for three directions. (c) Oriented point cloud $\mathcal{P}$, colored by tangent direction.

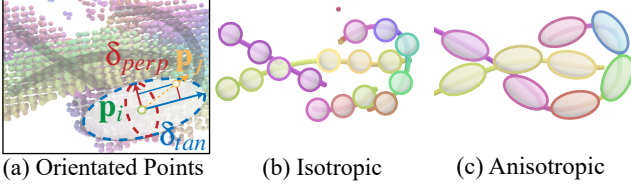


Figure 6: Anisotropic mean-shift kernel. (a) Oriented point cloud $\mathcal{P}$. (b) An isotropic kernel fuses neighboring passes where they run close together. (c) Our prolate kernel, elongated along the tangent, reaches far along the yarn axis and preserves its structure.

3.3. Center Point Set Extraction

The points in $\mathcal{P}$ are spread across the yarn volume, not on its centerline. We therefore iteratively project the point set toward the yarn centerline using a mean-shift procedure, which repeatedly moves each point toward the weighted average of its neighbors. Unlike conventional isotropic mean-shift, our method is anisotropic and exploits the local tangent direction at each point. First, neighboring points are weighted using a kernel elongated along the local tangent direction $\mathbf{t}_i$ (Fig. 6), so that the neighborhood extends farther along the yarn than across it. Second, the mean-shift update is projected onto the plane perpendicular to $\mathbf{t}_i$, preventing points from drifting along the yarn while allowing them to move toward the centerline.

Specifically, we define an anisotropic Gaussian kernel weight between points $\mathbf{p}_i$ and $\mathbf{p}_j$. Let $\mathbf{p}_j - \mathbf{p}_i$ be their relative offset. Decomposing this offset with respect to the local tangent $\mathbf{t}_i$ gives its component along the tangent, $\delta_{\text{tan}} = (\mathbf{p}_j - \mathbf{p}_i) \cdot \mathbf{t}_i$, and its component perpendicular to the tangent, $\delta_{\text{perp}} = \|(\mathbf{p}_j - \mathbf{p}_i) - \delta_{\text{tan}} \mathbf{t}_i\|$. The kernel weight is defined as

$$w_{ij} = \exp\left(-\frac{\delta_{\text{tan}}^2}{2\sigma_{\text{tan}}^2} - \frac{\delta_{\text{perp}}^2}{2\sigma_{\text{perp}}^2}\right) |\mathbf{t}_i \cdot \mathbf{t}_j|^\alpha. \quad (2)$$

The anisotropic bandwidths $\sigma_{\text{tan}} = \rho h$ and $\sigma_{\text{perp}} = h$ make the kernel elongated along the yarn direction, extending $\rho > 1$ times farther along the tangent than across it. The trailing factor up-weights neighbors whose tangent agrees with $\mathbf{t}_i$. The parameter settings are listed in Table 1.

We then move each point toward the weighted mean of its neighbors, with the shift projected onto the plane perpendicular to $\mathbf{t}_i$ so that only off-axis motion is applied:

$$\mathbf{p}_i \leftarrow \mathbf{p}_i + (\mathbf{I} - \mathbf{t}_i \mathbf{t}_i^\top) \left(\frac{\sum_j w_{ij} \mathbf{p}_j}{\sum_j w_{ij}} - \mathbf{p}_i \right). \quad (3)$$

Here the parenthesized term is the mean-shift vector toward the kernel-weighted average of the neighbors, and the projection $\mathbf{I} - \mathbf{t}_i \mathbf{t}_i^\top$ keeps only its component perpendicular to $\mathbf{t}_i$, so each point moves toward the centerline without drifting along the yarn. After each shift, we update the tangent direction by averaging neighboring tangents and renormalizing the result,

$$\mathbf{t}'_i = \frac{\sum_j w_{ij} \mathbf{t}_j}{\sum_j w_{ij}}, \quad \mathbf{t}_i \leftarrow \frac{\mathbf{t}'_i}{\|\mathbf{t}'_i\|} \quad (4)$$

which keeps the kernel aligned with the local yarn direction as the cloud tightens. Because the tangent represents an unoriented

local direction field, its sign is arbitrary. Before evaluating the weighted average in (4), we therefore flip the sign of $\mathbf{t}_j$ whenever $\mathbf{t}_j \cdot \mathbf{t}_i < 0$. Near endpoints and gaps, where the neighborhood becomes strongly one-sided, we attenuate the tangent update and retain the previous tangent estimate.

A conventional mean-shift procedure attracts points toward local density maxima. However, when applied to samples distributed along a yarn, this density-driven aggregation tends to collapse neighboring points into a small number of dense clusters, destroying the spatial ordering needed to recover the centerline. The neighborhood relations are still reliable during the early iterations, before substantial aggregation occurs. We therefore freeze these relations at an early stage and preserve them throughout the optimization using springs that penalize both compression and stretching relative to the distances recorded at the freeze stage. This regularization keeps neighboring points close to one another and prevents the centerline candidates from collapsing into disconnected clusters.

We progressively anneal the bandwidth, decreasing h geometrically from h_0 to h_T over T iterations. Because σ_{tan} and σ_{perp} are both proportional to h , they anneal jointly while their ratio ρ stays fixed: a large initial bandwidth pulls scattered points across noise and intermediate gaps, and the smaller final bandwidth tightens the cloud onto a thin centerline. The inner per-point loop of this procedure is data-parallel because each $\mathbf{p}_i$ reads only from its precomputed neighbors and writes only its own slot, so we run it on the GPU and complete a full mean-shift iteration over the entire cloud in milliseconds.

3.4. Automatic Topology Skeletonization

Fragment Extraction and Junction Detection The center points $\hat{\mathcal{P}}$ from mean-shift form an unordered cloud. We therefore organize them into ordered polylines. We first construct a proximity graph on $\hat{\mathcal{P}}$ by connecting pairs of points whose distance is below radius r_C , and discard connected components smaller than n_{min} points as residual noise. Within each remaining component we compute a minimum spanning tree (MST) and extract its diameter, i.e., the longest path between two leaves. This path typically follows the dominant yarn trajectory through the component and provides an ordering of its points. A component does not always correspond to a single clean yarn segment. MST nodes of degree ≥ 3 appear precisely where two yarn passes run close enough that the proximity graph links them across the contact, and the local structures they create are the ring, twig, and collapse patterns resolved in the junction cleaning and loop detection below. We therefore flag every such node as a junction. The output is a set of ordered fragments $\mathcal{F} = \{\mathcal{F}_1, \dots, \mathcal{F}_m\}$, each of which is an ordered polyline, together with the flagged junctions.

Fragment Linking The fragments themselves are still disconnected across the yarn-to-yarn gaps. To bridge them, we grow an ellipsoid at each fragment endpoint, elongated along the endpoint tangent so that it reaches farther along the yarn than across it, with its cross-section set by the proximity radius r_C . When a pair of endpoint ellipsoids overlap and neither end has any other overlapping partner, the connection is the unique local candidate and we link

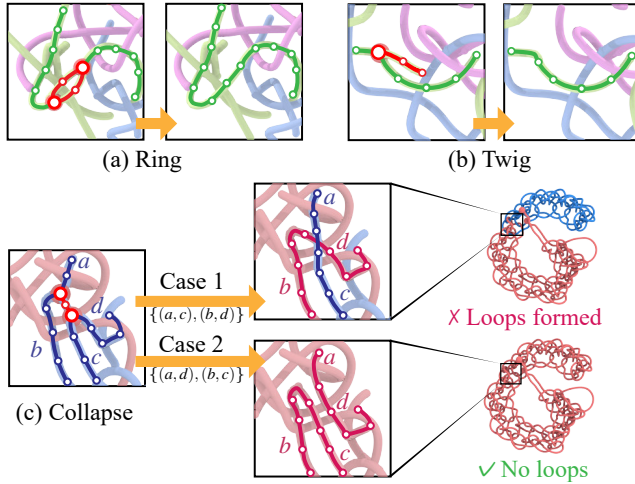


Figure 7: Three local patterns automatically resolved by our topology reconstruction. (a) Ring: a small closed loop inside a fragment. (b) Twig: a short spurious branch. (c) Collapse: two near-parallel passes wrongly merged.

the two fragments directly. Otherwise we keep each overlapping pairing only as a candidate bridge.

Junction Cleaning The flagged junctions from the fragment extraction fall into three local geometric patterns (Fig. 7). A *ring* is a small closed loop inside a fragment, where two junction nodes are joined by two distinct paths. We resample both paths and average their corresponding points into a single centerline that replaces the pair (Fig. 7a). A *twig* is a short spurious branch at a degree-three node: of the three branches meeting there, the one far shorter than the main path is pruned (Fig. 7b). Both patterns are resolved locally and without ambiguity. A *collapse* is where two near-parallel passes meet, forming a junction with four incident arms. We label them a, b, c, d , with a, b on one side of the junction and c, d on the other. A through-pass can never join two arms of the same side, so the four arms can be reconnected in only two ways, either $\{(a, c), (b, d)\}$ or $\{(a, d), (b, c)\}$, shown as Case 1 and Case 2 in Fig. 7c. The same-side pairing $\{(a, b), (c, d)\}$ is excluded because a collapse arises only where two near-parallel passes pinch together. A yarn that actually turned back to join a and b would form a single smooth curve on that side and would not produce a four-arm junction of this shape. Unlike the ring and twig, this choice cannot be made from local geometry alone, so we keep the collapse as an undecided connection with two cases.

Loop Detection To resolve the remaining undecided connections, the candidate bridges and the collapse cases, we exploit the single-strand structure of crochet. A physically crocheted sample is wound from one continuous strand, so its true centerline is topologically an open path: one connected component, two endpoints, and no cycle. Any cycle in the reconstructed topology therefore implies a wrong connection, since the correct centerline cannot revisit a point along its own trajectory. Rather than resolving ambiguous junctions locally, we retain all possible connection choices. Each collapse contributes two cases and each candidate bridge two states, so N

undecided connections give 2^N candidate topologies. We enumerate them and select the unique configuration that forms a single open path as the ordered yarn path $\mathcal{C}$. Only the correct configuration keeps every fragment on one continuous centerline without a loop, which for a collapse means all four arms a, b, c, d stay on the same strand (Fig. 7c). The enumeration is tractable in practice: N peaks at five across our dataset, and branches are pruned early because a single closed cycle already invalidates every configuration that contains it. When no candidate forms a single open path, a wrong connection must have been introduced by an earlier step, which we cannot resolve automatically, so all offending junctions are passed to the manual repair stage (Sec. 3.5). Once the topology is fixed, automatically or after manual repair, we fit a B-spline of smoothing factor s_C through its ordered center points to obtain the smooth final centerline.

3.5. Manual Repair

While the automatic stage is designed to recover the complete yarn path, crochet regions with complex topology can still produce highly ambiguous breaks or missing connections that no automatic rule resolves reliably. As a fallback for these cases, we provide a lightweight interactive interface built on Polyscope [Sha*19] for manual repair, offering two ways to connect two broken ends, a Bézier link and a freehand sketch. As Fig. 8a shows, the user sees a 3D view of the current topology. The user picks two nodes in the current 3D view, and we fit a 2D Bézier curve between them in screen space. Given the tangents at the two picked nodes, the control points can be computed by intersecting these tangents, as Fig. 8b shows, and the user can also adjust them to specify a smooth 2D path between the two endpoints. Alternatively, for connecting

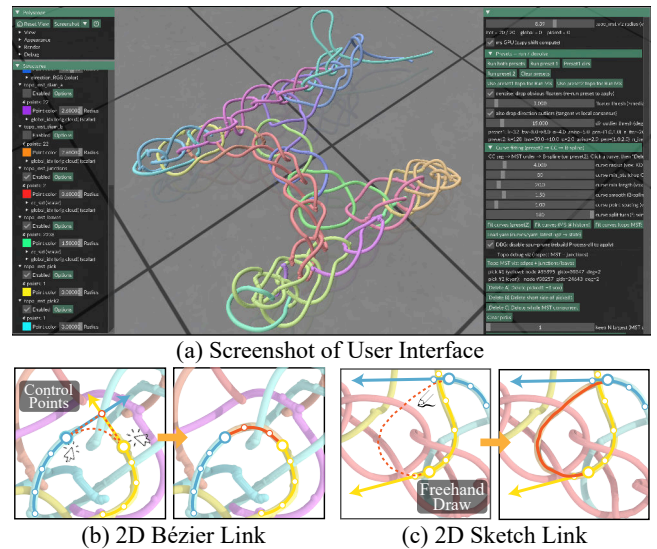


Figure 8: Manual editing interface. (a) Topology graph overlaid on the 3D view. (b) Bézier mode: the user shapes a smooth path between two picked endpoints with control handles. (c) Sketch mode: the user draws a freehand stroke between them. Either 2D curve is back-projected to 3D as a bridge.

Symbol	Meaning	Value
<i>Input</i>		
$\mathcal{V}$	CT volume	—
<i>Orientation field</i>		
K	Gabor carrier directions (Fibonacci)	128
σ_{Gabor}	Gabor envelope and tangent-smoothing width (vox)	7.0
ω	Gabor frequency	0.05
τ_E	Per-direction response threshold	250
$\mathbf{t}(\mathbf{x})$	Per-voxel tangent vector	—
$\mathcal{P}$	Oriented point cloud	—
<i>Center point set extraction</i>		
ρ	Anisotropy ratio	2.0
α	Direction power	2
h_0, h_T	Bandwidth schedule (vox)	30 $\rightarrow$ 10
T	Mean-shift iterations	20
σ_{tan}	Kernel width along tangent	ρh
σ_{perp}	Kernel width across tangent	h
$\tilde{\mathcal{P}}$	Center points (post mean-shift)	—
<i>Topology Skeletonization & Manual Repair</i>		
r_C	Proximity radius (vox)	4.0
n_{min}	Min. component size (pts)	30
s_C	B-spline smoothing factor	1.5
$\mathcal{F}$	Yarn fragments	—
$\mathcal{C}$	Output yarn curves	—
<i>Stitch fitting</i>		
M	Sample points per stitch	30
S_I	Canonical stitch template	—
S_C	Candidate stitch (centerline window)	—
e^*	Refined stitch end point	—

Table 1: Key symbols used in the method together with the parameter values used in our experiments. The same configuration is applied to every sample across the whole dataset.

nodes that are far apart, the user can directly sketch a freehand stroke between the two picks in screen space, as Fig. 8c shows. Each pixel along that 2D curve is back-projected to 3D by linearly interpolating its depth between the two picks, yielding a bridge curve that passes through both endpoints, matches the drawn shape from the chosen viewpoint, and is inserted into the topology graph as an additional segment. After user finish all edits, we rerun the loop detection of the automatic stage on the updated topology, verifying that the repaired centerline remains a valid single open path.

4. Experiments

Implementation Details The pipeline is implemented in Python with GPU kernels in PyTorch. All measurements reported in this section are performed on a workstation with an AMD Ryzen 9 9950X (16-core / 32-thread, 5.0 GHz), 128 GB of system memory, and a single NVIDIA GeForce RTX 3090 (24 GB VRAM). Table 1 lists the symbols used throughout the method together with the numerical values we use in our experiments. The same set is used for every sample unless otherwise noted. Most parameters are physically grounded. The yarn radius R read from a CT slice sets the Gabor width σ_{Gabor} , frequency ω , and mean-shift bandwidths $h_0 \rightarrow h_T$, so adapting to a different sample or scanner

mainly amounts to re-measuring the yarn radius voxels. The response threshold τ_E is defined per Gabor direction and is thus independent of the number of directions K . The remaining parameters, those of the center point extraction and topology stages, are chosen empirically. Regarding the choice of K , the recovered geometry is insensitive to it, with the Chamfer distance varying within 0.13 voxels for K from 32 to 256. Larger K sharpens the tangents at higher cost, and we use $K=128$ as the trade-off.

4.1. Dataset

Acquisition. We collected a dataset of 18 crocheted samples imaged on a Rigaku HX Lab130 high-resolution micro-CT scanner. Each volume is reconstructed at full resolution, spanning roughly 900 to 2200 voxels in the two in-plane axes and 300 to 550 slices along the scan axis, and stored as a single 16-bit Nearly Raw Raster Data (NRRD) file. To isolate geometric variability from material variability, all samples are crocheted from the same recycled cotton yarn intended for 3.0–4.0 mm crochet hooks. We additionally synthesize a volume with a known centerline by simulating a yarn model along the single-yarn path of the $3 \times 3 \times 3$ solid-knit cube of Hirose et al. [HGBM24]. The model twists eight plies of 40 fibers each into a yarn of radius 15 voxels, which we rasterize into a density volume and degrade with a Gaussian point-spread function, distinct air and yarn intensity distributions, and spatially correlated noise, so that the contrast between touching yarn passes is as weak as in our real scans.

Composition. The 18 samples fall into five categories spanning the structural variety in real crochet pieces: PATCHES (3, single-stitches crocheted patches), CHAIN STITCHES (3, the simplest chain stitch patterns), LETTERS (6, stitched letter forms with more complex junctions), NUMBERS (4, including the digit two at three different tensions and the digit six)[†], and MISC (2, stick and arrow shaped crochet). The number in brackets refers to the number of samples in each category.

[†] The crocheted LETTERS and NUMBERS patterns follow designs by amiozi, <https://amiozi.myportfolio.com/>.

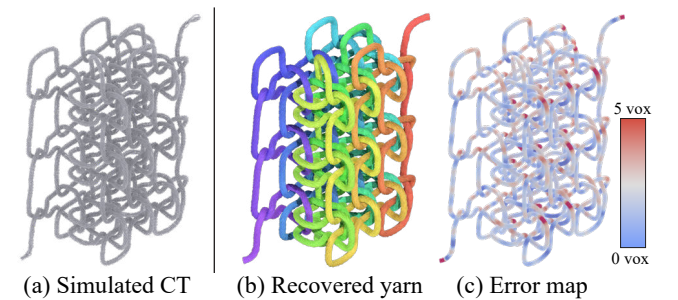


Figure 9: Validation on synthetic data. (b) The centerline our pipeline recovers from the synthetic volume (a), colored by arc length, and (c) its distance to the exact ground truth, colored from 0 to 5 voxels.

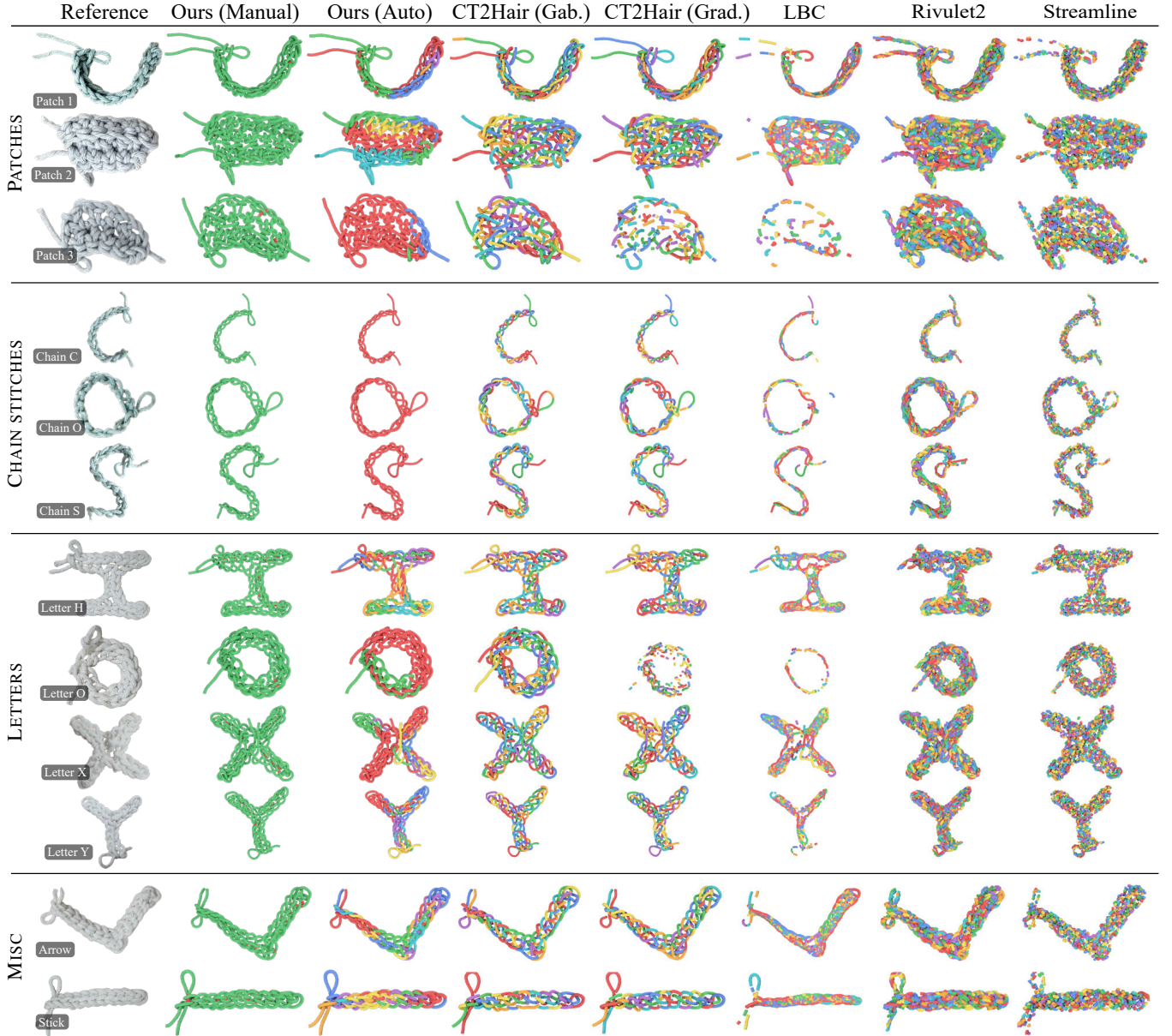


Figure 10: Visual comparison against five baselines on twelve samples grouped by category. Reference shows a photograph of the physical sample. In Ours (Manual), green marks the automatically recovered yarn and red the parts completed by interactive sketch edits. All other columns are colored by fragment identity, one color per recovered curve.

4.2. Metrics

The goal of reconstruction is a yarn path that is visually and topologically consistent with the scanned object, since the downstream applications depend on this overall consistency rather than on sub-voxel geometric agreement. Our evaluation therefore rests primarily on visual comparison (Fig. 10), with quantitative metrics as supporting evidence. As no ground-truth yarn geometry is available for real crocheted samples, the metrics are computed against a *reference* yarn path that we manually annotate for each sample using the interactive editor, and all results are averaged over the 18 samples.

Two metrics are *reference-free*: the number of recovered curves $\#C$ (ideally 1, a single continuous yarn) and the mean recovered-curve arc-length $\overline{L_C}$ in voxels, which together describe how fragmented the output is. The remaining four are *reference-based*: the relative total-length error $E_L = |L/L_{\text{ref}} - 1|$, the bidirectional Chamfer distance ℓ_{CD} between the recovered and reference point sets, the mean distance ℓ_2 from each reference point to its nearest recovered point, both in voxels, and the median angle $\tilde{\alpha}$ between matched tangents. Each metric is blind to a different failure mode, so no single number should be read in isolation.

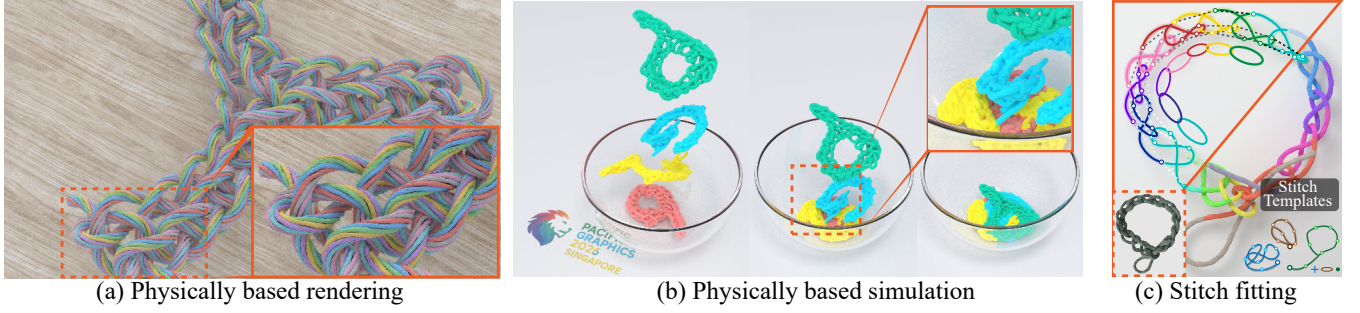


Figure 11: Downstream applications of the recovered yarn path. (a) Physically based ply-level rendering in Mitsuba 3. (b) Physically based simulation as a Discrete Elastic Rod. (c) Stitch pattern fitting via template matching.

4.3. Validation on Synthetic Data

To complement the manually annotated references with an absolute ground truth, we run our pipeline on the synthetic volume described in Sec. 4.1, whose exact centerline is known by construction. The automatic stage alone recovers a single continuous centerline here, without any interactive edit, at a total-length error of 1.1% and a Chamfer distance of 3.45 voxels, or 0.23 yarn radii. As Fig. 9 shows, the deviation stays below a third of the yarn radius along 95% of the path and grows only at the two free yarn ends, where the reconstruction stops short of the true tips. The accuracy observed against our manual annotations is therefore consistent with the accuracy against an exact ground truth.

4.4. Comparison to Baselines

We compare against five baselines on our crochet dataset. Two are CT2Hair-derived pipelines [SSW*23]: *Gabor* feeds our Gabor-oriented point cloud into the mean-shift stage of CT2Hair, while *Gradient* keeps that pipeline but replaces the Gabor field with the smoothed gradient field of CT2Hair. The remaining three operate on different principles. Laplacian-based contraction [CTO*10] contracts the point cloud onto a one-dimensional skeleton by repeated Laplacian smoothing. The Rivulet2 neuron tracer [LZS*18] back-traces tubular structures directly from the volume along intensity ridges. Streamline tracing integrates curves by following the dense per-voxel orientation field.

Figure 10 shows the comparison on twelve samples. Our reconstruction reproduces the yarn structure faithfully on every sample, whereas each baseline exhibits a characteristic failure such as fragmentation, missing arcs, parallel-rail duplication, or over-contraction. The quantitative results in Table 2, averaged over all 18 samples, agree with this visual impression. Our automatic pipeline returns by far the fewest curves and the longest mean arc-length, and it leads on the reference-based metrics as well. Its average of 15 curves per sample is still above the ideal of a single curve, so the automatic output is a strong starting point rather than a finished reconstruction. The interactive effort that closes this gap is small. All 18 samples reach a single continuous yarn path, with a median of 4 sketch edits and 2:46 of editing (mean 7.5 edits, 3:49), and the edited strand contributes 2.0% of the final centerline length on average, 1.0% at the median. The three chain-stitch samples need no editing at all.

Method	Reference-free		Reference-based				Effort	
	#C	$\overline{L_C}$	$E_L \downarrow$	$\tilde{\alpha} \downarrow$	$\ell_{CD} \downarrow$	$\ell_2 \downarrow$	edits	time
CT2Hair (Gab.)	88	314	0.115	10.55	16.99	18.55	—	—
CT2Hair (Grad.)	115	236	0.271	12.75	20.60	24.67	—	—
LBC	191	92	0.456	53.94	31.92	44.29	—	—
Rivulet2	676	140	2.141	35.53	23.31	17.53	—	—
Streamline	730	49	0.230	25.05	21.32	19.41	—	—
Ours (Auto)	15	3814	0.002	2.66	2.43	2.26	—	—
Ours (Manual)	1	30359	—	—	—	—	4	2:46

Table 2: Quantitative comparison to baselines. Averaged over the 18 samples, with metrics defined in Sec. 4.2 and arrows marking the better direction. Effort is the median number of sketch edits and editing time (min:s) per sample. Our automatic output already leads every baseline, and a few edits complete it into a single continuous yarn on all 18 samples.

4.5. Ablation Study

We ablate the components of the center-point extraction stage (Sec. 3.3) across all 18 samples in our dataset. *Full* is the complete center-extraction pipeline and each remaining variant drops one mean-shift component: the anisotropic kernel (reverting to an isotropic ball), the per-iteration direction update, and the topology-preservation step. The anisotropic kernel matters most. Replacing it with an isotropic ball inflates the ℓ_{CD} and ℓ_2 errors by roughly 40%, raises the tangent error, and fragments the cloud into half-again as many curves, because a round kernel pulls neighboring

Variant	Reference-free		Reference-based			
	#C	$\overline{L_C}$	$E_L \downarrow$	$\tilde{\alpha} \downarrow$	$\ell_{CD} \downarrow$	$\ell_2 \downarrow$
w/o Anisotropic MS	22	2511	0.022	3.46	3.45	3.14
w/o Direction Update	19	2703	0.024	3.03	2.85	2.65
w/o Topology	20	2155	0.003	2.69	2.54	2.34
Ours (Full)	15	3814	0.002	2.66	2.43	2.26

Table 3: Center-extraction ablation. This table shows the performance of different variants of our center-extraction method, averaged over all samples. Arrows mark the better direction.

passes together instead of following each strand. Dropping the per-iteration direction update degrades the geometry more mildly, and removing topology preservation leaves the point-wise metrics almost unchanged while raising the curve count, confirming that its effect is mainly on connectivity rather than point placement.

4.6. Applications for the Extracted Centerline

Simulation and Rendering To produce ply-level geometry we wrap the recovered centerline with the coaxial-helix procedural ply model of Zhao et al. [ZLB16], yielding a twisted thread that follows the centerline. We render this geometry in Mitsuba 3 [JSR*22] with a custom shader that adds Perlin noise [SZK14] along the ply axis for the fiber-level fuzz visible at the yarn surface (Fig. 11a). For physically based simulation we treat the centerline as a discrete elastic rod [BWR*08] and integrate it under gravity and self-contact (Fig. 11b).

Stitch Pattern Extraction Once the centerline $\mathcal{C}$ has been recovered, we decompose it into an ordered sequence of stitches $\mathcal{S} = \{\mathcal{S}_1, \dots, \mathcal{S}_n\}$, where each stitch $\mathcal{S}_i$ is an interval of $\mathcal{C}$ tagged with a stitch type. The user marks where the first stitch $\mathcal{S}_1$ starts. We then sweep a window along the centerline and label every subsequent stitch by matching it against a small retrieval library of three canonical templates: a chain stitch, a slip stitch, and a single stitch, shown in Fig 12. For a candidate stitch $\mathcal{S}_c$ ending at e , we sample M points uniformly along both $\mathcal{S}_c$ and each canonical template $\mathcal{S}_t$ in the library $\{\mathcal{S}_{ch}, \mathcal{S}_{sl}, \mathcal{S}_{sg}\}$, and rigidly align the two point sets with the Kabsch algorithm [Kab76] by measuring the root mean square deviation (RMSD) between them. After alignment, the per-template residual is

$$\text{RMSD}(\mathcal{S}_c, \mathcal{S}_t) = \sqrt{\frac{1}{M} \sum_{k=1}^M \|\mathcal{S}_c(k) - \mathcal{S}_t(k)\|^2}, \quad (5)$$

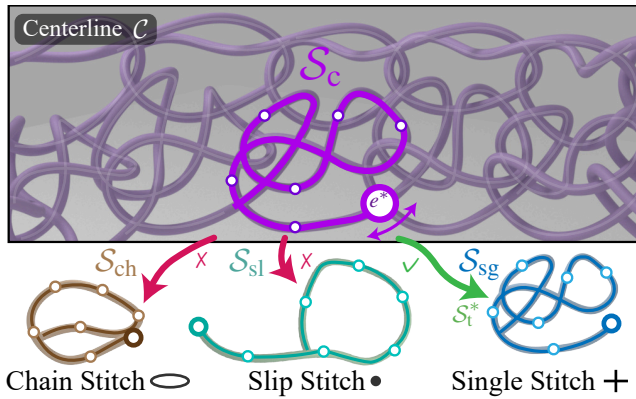


Figure 12: Stitch pattern fitting. From a seed on the recovered centerline, each candidate window is rigidly aligned to three stitch-type templates (bottom: chain, slip, single) with the Kabsch algorithm. $\mathcal{S}_t^*$ is the template that yields the smallest RMSD. e^* is the refined end point of that best-fit stitch, which becomes the start anchor of the next iteration.

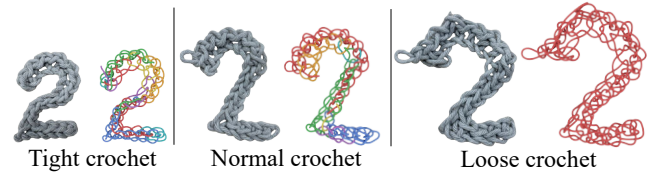


Figure 13: Limitation on tightly crocheted samples. Reconstruction results for identical crochet patterns made with different stitch tensions: tight (left), normal (middle), and loose (right). Different colors denote different recovered fragments. Higher stitch tension increases contact between neighboring yarn segments, leading to shorter and more fragmented reconstructions.

where $\mathcal{S}_c(k)$ and $\mathcal{S}_t(k)$ denote the k -th of the M aligned sample points. We label $\mathcal{S}_c$ with the template $\mathcal{S}_t^*$ of smallest RMSD. To refine the boundary, we slide the end point e forward and backward within a small neighborhood and take $e^* = \arg \min_e \text{RMSD}(\mathcal{S}_c, \mathcal{S}_t^*)$, the location that best fits the chosen type. This e^* closes the fitted stitch $\mathcal{S}_i$ and becomes the start anchor of the next iteration.

5. Conclusion

We presented CT2Yarn, a human-in-the-loop framework that recovers a single continuous yarn path from the micro-CT density volume of a real crocheted object. Because the fiber orientations observed in the volume are not aligned with the yarn direction, we estimate a voxel-wise orientation field with Gabor filtering and aggregate it into yarn-level orientations through an anisotropic mean-shift procedure. A topology-preserving skeletonization then extracts yarn-path fragments as a set of disjoint trees, which a heuristic graph-repair stage merges into long continuous sections by exploiting the single-yarn constraint of crochet. For the connections the automatic pass cannot resolve, a graphical user interface enables interactive correction.

We evaluated our method on micro-CT scans of 18 hand-crocheted samples covering three stitch types, primarily through visual comparison, supported by quantitative metrics against manually annotated references and against a synthetic volume with an exact ground truth, and by an ablation study of our design choices. The recovered yarn path supports downstream applications including physically based simulation, ply-level rendering, and stitch-pattern extraction, and we release both the micro-CT volumes and the reconstructed centerlines to support future research.

Limitations and Future Work Our pipeline degrades on tightly crocheted samples. When stitches are pulled taut, adjacent yarns press together and the CT contrast at their boundary drops to the noise level, so the mean-shift stage loses its separation cue and merges center points that should stay distinct. As Fig. 13 shows, for the same stitch pattern the way it is crocheted strongly affects the extracted centerline: loose samples are reconstructed without any edits, whereas the curves in tight samples are much shorter and require further manual editing.

Our stitch-pattern extraction is also limited to approximately planar pieces and a small set of basic stitch types, and extending it

to non-planar layouts and a richer stitch vocabulary is left for future work. A further challenge is to generalize the pipeline to fully three-dimensional crocheted objects such as amigurumi. The single continuous yarn of our samples is not merely an input assumption but the topological constraint behind our loop detection, whereas amigurumi are typically assembled from several separate yarns and stuffed with a fibrous filling that is difficult to distinguish from the yarn in CT. Reconstructing them would require reasoning about multiple interacting strands and separating yarn from filling.

Finally, our semi-automatic workflow produces paired CT volumes and yarn centerlines with little human effort, so it can generate training data at scale. Such a dataset could support learned centerline detectors for harder, lower-contrast scans, as well as downstream applications such as centerline-aware crochet pattern generation and neural stitch classification.

Acknowledgements

We would like to thank Yumi Endo for her assistance with crafting crochet, as well as the anonymous reviewers for their constructive comments, which helped improve the manuscript. This work was supported by JST SPRING, Grant Number JPMJSP2108.

References

- [BWR*08] BERGOU, MIKLÓS, WARDETSKY, MAX, ROBINSON, STEPHEN, et al. “Discrete elastic rods”. *ACM SIGGRAPH 2008 Papers*. SIGGRAPH ’08. Los Angeles, California: Association for Computing Machinery, 2008. ISBN: 9781450301121. DOI: [10.1145/1399504.1360662](https://doi.org/10.1145/1399504.1360662) 10.
- [ÇBG17] ÇAPUNAMAN, ÖZGÜÇ BERTUĞ, BİNGÖL, CEMAL KORAY, and GÜRSOY, BENAY. “Computing Stitches and Crocheting Geometry”. *Computer-Aided Architectural Design. Future Trajectories*. Ed. by ÇAĞDAŞ, GÜLEN, ÖZKAR, MINE, GÜL, LEMAN FİGEN, and GÜRER, ETHEM. Vol. 724. Communications in Computer and Information Science. CAAD Futures 2017. Singapore: Springer, 2017, 289–305. DOI: [10.1007/978-981-10-5197-5_16](https://doi.org/10.1007/978-981-10-5197-5_16) 3.
- [CLO17] CIRIO, GABRIEL, LOPEZ-MORENO, JORGE, and OTADUY, MIGUEL A. “Yarn-Level Cloth Simulation with Sliding Persistent Contacts”. *IEEE Transactions on Visualization and Computer Graphics* 23.2 (2017), 1152–1162. DOI: [10.1109/TVCG.2016.2592908](https://doi.org/10.1109/TVCG.2016.2592908) 3.
- [CTO*10] CAO, JUNJIE, TAGLIASACCHI, ANDREA, OLSON, MATT, et al. “Point Cloud Skeletons via Laplacian Based Contraction”. *2010 Shape Modeling International Conference*. 2010, 187–197. DOI: [10.1109/SMI.2010.259](https://doi.org/10.1109/SMI.2010.259).
- [CWW24] CHEN, XIANG, WANG, LU, and WANG, BEIBEI. “Real-Time Neural Woven Fabric Rendering”. *ACM SIGGRAPH 2024 Conference Papers*. SIGGRAPH ’24. Denver, CO, USA: Association for Computing Machinery, 2024. ISBN: 9798400705250. DOI: [10.1145/3641519.3657496](https://doi.org/10.1145/3641519.3657496) 3.
- [EPIB22] EDELSTEIN, MICHAL, PELEG, HILA, ITZHAKY, SHACHAR, and BEN-CHEN, MIRELA. “AmiGo: Computational Design of Amigurumi Crochet Patterns”. *Proceedings of the 7th Annual ACM Symposium on Computational Fabrication*. SCF ’22. Seattle, WA, USA: Association for Computing Machinery, 2022. ISBN: 9781450398725. DOI: [10.1145/3559400.3562005](https://doi.org/10.1145/3559400.3562005) 2, 3.
- [FDK84] FELDKAMP, LEE A, DAVIS, LLOYD C, and KRESS, JAMES W. “Practical cone-beam algorithm”. *Journal of the Optical Society of America A* 1.6 (1984), 612–619 4.
- [GHCG17] GUARNERA, GIUSEPPE CLAUDIO, HALL, PETER, CHESNAIS, ALAIN, and GLENCROSS, MASHHUDA. “Woven Fabric Model Creation from a Single Image”. *ACM Trans. Graph.* 36.5 (Oct. 2017). ISSN: 0730-0301. DOI: [10.1145/3132187](https://doi.org/10.1145/3132187) 3.
- [GLNM20] GUO, RUNBO, LIN, JENNY, NARAYANAN, VIDYA, and MCCANN, JAMES. “Representing Crochet with Stitch Meshes”. *Proceedings of the 5th Annual ACM Symposium on Computational Fabrication*. SCF ’20. Virtual Event, USA: Association for Computing Machinery, 2020. ISBN: 9781450381703. DOI: [10.1145/3424630.3425409](https://doi.org/10.1145/3424630.3425409) 2, 3.
- [GOS09] GONG, R. HUGH, OZGEN, BANU, and SOLEIMANI, MANUCHEHR. “Modeling of Yarn Cross-Section in Plain Woven Fabric”. *Textile Research Journal* 79.11 (2009), 1014–1020. DOI: [10.1177/0040517508101799](https://doi.org/10.1177/0040517508101799) 3.
- [HGBM24] HIROSE, YUICHI, GILLESPIE, MARK, BONILLA FOMINAYA, ANGELICA M., and MCCANN, JAMES. “Solid Knitting”. *ACM Trans. Graph.* 43.4 (July 2024). DOI: [10.1145/3658123](https://doi.org/10.1145/3658123) 7.
- [HLI26] HIROSE, YUICHI, LARSSON, MARIA, and IGARASHI, TAKEO. “skCAD: A Design Tool for Solid Knitting with Automatic Pattern Generation”. *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. CHI ’26. Barcelona, Spain: Association for Computing Machinery, 2026. ISBN: 9798400722783. DOI: [10.1145/3772318.3791148](https://doi.org/10.1145/3772318.3791148) 3.
- [HLN*24] HAN, SHUAIKANG, LI, QIANHE, NEWTON, MD ALL AMIN, et al. “Research on Cotton Yarn Based on Synchrotron Radiation 3D Micro-CT Imaging”. *Fibers and Polymers* 25.2 (2024), 543–555 3.
- [HSW*25] HUANG, TAO, SHI, HAORYANG, WANG, MENGDI, et al. “Real-Time Knit Deformation and Rendering”. *ACM Trans. Graph.* 44.4 (July 2025). ISSN: 0730-0301. DOI: [10.1145/3731184](https://doi.org/10.1145/3731184) 2, 3.
- [IIS08] IGARASHI, YUKI, IGARASHI, TAKEO, and SUZUKI, HIROMASA. “Knitty: 3D Modeling of Knitted Animals with a Production Assistant Interface”. *Eurographics (Short Papers)*. 2008, 17–20 3.
- [JSR*22] JAKOB, WENZEL, SPEIERER, SÉBASTIEN, ROUSSEL, NICOLAS, et al. *Mitsuba 3 renderer*. Version 3.0.1. <https://mitsuba-renderer.org>. 2022 10.
- [JWH*22] JIN, WENHUA, WANG, BEIBEI, HASAN, MILOS, et al. “Woven Fabric Capture from a Single Photo”. *SIGGRAPH Asia 2022 Conference Papers*. SA ’22. Daegu, Republic of Korea: Association for Computing Machinery, 2022. ISBN: 9781450394703. DOI: [10.1145/3550469.3555380](https://doi.org/10.1145/3550469.3555380) 2, 3.
- [JZY*26] JABBAR, ABDUL, ZHANG, HAOQI, YANG, DONGMIN, et al. “Micro-computed Tomography Analysis of Ring, Rotor, and Air Jet Yarn Structures and Their Impact on Fragmented Fibres Release from Polyester Textiles”. *Nondestructive Testing and Evaluation* 41.4 (2026), 1767–1788. DOI: [10.1080/10589759.2025.2482868](https://doi.org/10.1080/10589759.2025.2482868) 3.
- [Kab76] KABSCH, WOLFGANG. “A solution for the best rotation to relate two sets of vectors”. *Acta Crystallographica Section A* 32.5 (1976), 922–923. DOI: [10.1107/S0567739476001873](https://doi.org/10.1107/S0567739476001873) 10.
- [KJM08] KALDOR, JONATHAN M., JAMES, DOUG L., and MARSHNER, STEVE. “Simulating Knitted Cloth at the Yarn Level”. *ACM SIGGRAPH 2008 Papers*. SIGGRAPH ’08. Los Angeles, California: Association for Computing Machinery, 2008. ISBN: 9781450301121. DOI: [10.1145/1399504.1360664](https://doi.org/10.1145/1399504.1360664) 2, 3.
- [KOM*19] KASPAR, ALEXANDRE, OH, TAE-HYUN, MAKATURA, LIANE, et al. “Neural Inverse Knitting: From Images to Manufacturing Instructions”. *Proceedings of the 36th International Conference on Machine Learning*. Ed. by CHAUDHURI, KAMALIKA and SALAKHUTDINOV, RUSLAN. Vol. 97. Proceedings of Machine Learning Research. Long Beach, California, USA: PMLR, June 2019, 3272–3281. URL: <http://proceedings.mlr.press/v97/kaspar19a.html> 2, 3.
- [KSZ*16] KHUNGURN, PRAMOOK, SCHROEDER, DANIEL, ZHAO, SHUANG, et al. “Matching Real Fabrics with Micro-Appearance Models”. *ACM Trans. Graph.* 35.1 (2016). ISSN: 0730-0301. DOI: [10.1145/2818648](https://doi.org/10.1145/2818648) 3.
- [KWA11] KNUTSSON, HANS, WESTIN, CARL-FREDRIK, and ANDERSSON, MATS. “Representing Local Structure Using Tensors II”. *Scandinavian Conference on Image Analysis*. Springer. Springer, 2011, 545–556 4.

- [LHHC25] LI, PEIYU, HUANG, XIAOBAO, HUA, TING, and CHAWLA, NITESH V. *CrochetBench: Can Vision-Language Models Move from Describing to Doing in Crochet Domain?* 2025. arXiv: 2511.09483 [cs.AI]. URL: <https://arxiv.org/abs/2511.09483> 2.
- [LSS*26] LI, ZIXUAN, SHEN, PENGFEI, SUN, HANXIAO, et al. “Fiber-Level Woven Fabric Capture from a Single Microscopic Image”. *ACM Trans. Graph.* (May 2026). Just Accepted. ISSN: 0730-0301. DOI: 10.1145/3816036 2, 3.
- [LWS*18] LEAF, JONATHAN, WU, RUNDONG, SCHWEICKART, ESTON, et al. “Interactive design of periodic yarn-level cloth patterns”. *ACM Trans. Graph.* 37.6 (Dec. 2018). ISSN: 0730-0301. DOI: 10.1145/3272127.3275105 3.
- [LZS*18] LIU, SIQI, ZHANG, DONGHAO, SONG, YANG, et al. “Automated 3-D neuron tracing with precise branch erasing and confidence controlled back tracking”. *IEEE transactions on medical imaging* 37.11 (2018), 2441–2452 9.
- [MAN*16] MCCANN, JAMES, ALBAUGH, LEA, NARAYANAN, VIDYA, et al. “A Compiler for 3D Machine Knitting”. *ACM Trans. Graph.* 35.4 (July 2016). ISSN: 0730-0301. DOI: 10.1145/2897824.2925940 3.
- [MLB25] MARCINIAK, ZOFIA, LERTJATURAPHAT, PUNN, and BIANCHI, ANDREA. “StitchFlow: Enabling In-Situ Creative Explorations of Crochet Patterns with Stitch Tracking and Process Sharing”. *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. UIST ’25. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400720376. DOI: 10.1145/3746059.3747715 2, 3.
- [NWKS19] NAM, GILJOO, WU, CHENGLEI, KIM, MIN H., and SHEIKH, YASER. “Strand-Accurate Multi-View Hair Capture”. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019, 155–164 3.
- [NWYM19] NARAYANAN, VIDYA, WU, KUI, YUKSEL, CEM, and MCCANN, JAMES. “Visual Knitting Machine Programming”. *ACM Trans. Graph.* 38.4 (July 2019). ISSN: 0730-0301. DOI: 10.1145/3306346.3322995 3.
- [Ots79] OTSU, NOBUYUKI. “A Threshold Selection Method from Gray-Level Histograms”. *IEEE Transactions on Systems, Man, and Cybernetics* 9.1 (1979), 62–66. DOI: 10.1109/TSMC.1979.4310076 4.
- [SFFK23] STORCK, JAN LUKAS, FELDMANN, BJARTE ALEXANDER, FIEDLER, JOHANNES, and KYOSEV, YORDAN. “Numerical Optimization of Polygon Tessellation for Generating Machine-Productible Crochet Patterns”. *Tekstilec* 66.4 (2023), 263–284. DOI: 10.14502/tekstilec.66.2023062 3.
- [SFK23] STORCK, JAN LUKAS, FELDMANN, BJARTE ALEXANDER, and KYOSEV, YORDAN. “Design Tool for Automated Crocheting of Fabrics”. *Communications in Development and Assembling of Textile Products* 4.2 (2023), 254–272. DOI: 10.25367/cdatp.2023.4.p254-272 3.
- [SGSK22] STORCK, JAN LUKAS, GERBER, DENNIS, STEENBOCK, LISKA, and KYOSEV, YORDAN. “Topology Based Modelling of Crochet Structures”. *Journal of Industrial Textiles* 52 (2022), 15280837221139250. DOI: 10.1177/15280837221139250 3.
- [Sha*19] SHARP, NICHOLAS et al. *Polyscope*. www.polyscope.run. 2019 6.
- [SNW20] SPERL, GEORG, NARAIN, RAHUL, and WOJTAN, CHRIS. “Homogenized Yarn-Level Cloth”. *ACM Trans. Graph.* 39.4 (Aug. 2020). ISSN: 0730-0301. DOI: 10.1145/3386569.3392412 3.
- [SRLH22] SEITZ, KLARA, REIN, PATRICK, LINCKE, JENS, and HIRSCHFELD, ROBERT. “Digital Crochet: Toward a Visual Language for Pattern Description”. *Proceedings of the 2022 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. Onward! 2022. Auckland, New Zealand: Association for Computing Machinery, 2022, 48–62. ISBN: 9781450399098. DOI: 10.1145/3563835.3567657 3.
- [SSW*23] SHEN, YUEFAN, SAITO, SHUNSUKE, WANG, ZIYAN, et al. “CT2Hair: High-Fidelity 3D Hair Modeling using Computed Tomography”. *ACM Trans. Graph.* 42.4 (July 2023). ISSN: 0730-0301. DOI: 10.1145/3592106 2-4, 9.
- [STOK10] SHINOHARA, TOSHIHIRO, TAKAYAMA, JUN-YA, OHYAMA, SHINJI, and KOBAYASHI, AKIRA. “Extraction of Yarn Positional Information from a Three-dimensional CT Image of Textile Fabric using Yarn Tracing with a Filament Model for Structure Analysis”. *Textile Research Journal* 80.7 (2010), 623–630. DOI: 10.1177/0040517509342320 3.
- [SZK14] SCHRÖDER, KAI, ZINKE, ARNO, and KLEIN, REINHARD. “Image-based reverse engineering and visual prototyping of woven cloth”. *IEEE transactions on visualization and computer graphics* 21.2 (2014), 188–200 10.
- [SZK15] SCHRÖDER, KAI, ZINKE, ARNO, and KLEIN, REINHARD. “Image-Based Reverse Engineering and Visual Prototyping of Woven Cloth”. *IEEE Transactions on Visualization and Computer Graphics* 21.2 (Feb. 2015), 188–200. DOI: 10.1109/TVCG.2014.2339831 3.
- [TLH*24] TANG, YINGJIE, LI, ZIXUAN, HAŠAN, MILOŠ, et al. “Woven Fabric Capture with a Reflection-Transmission Photo Pair”. *ACM SIGGRAPH 2024 Conference Papers*. SIGGRAPH ’24. Denver, CO, USA: Association for Computing Machinery, 2024. ISBN: 9798400705250. DOI: 10.1145/3641519.3657410 3.
- [TMK*19] TRUNZ, ELENA, MERZBACH, SEBASTIAN, KLEIN, JONATHAN, et al. “Inverse Procedural Modeling of Knitwear”. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019, 8622–8631. DOI: 10.1109/CVPR.2019.00883 3.
- [WGF*18] WU, KUI, GAO, XIFENG, FERGUSON, ZACHARY, et al. “Stitch Meshing”. *ACM Trans. Graph.* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201360 3.
- [WKZ*25] WU, CHENGHAO, KHATTAR, APOORV, ZHU, JUNQIU, et al. “Automatic Reconstruction of Woven Cloth from a Single Close-up Image”. *Computer Graphics Forum* 44.7 (2025). Pacific Graphics 2025 proceedings issue, e70243. DOI: 10.1111/cgfm.70243. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgfm.70243> 3.
- [WSY19] WU, KUI, SWAN, HANNAH, and YUKSEL, CEM. “Knittable Meshes”. *ACM Trans. Graph.* 38.1 (Jan. 2019). ISSN: 0730-0301. DOI: 10.1145/3292481 2, 3.
- [WY19] WU, KUI and YUKSEL, CEM. “Real-Time Cloth Rendering with Fiber-Level Detail”. *IEEE Transactions on Visualization and Computer Graphics* 25.2 (2019), 1297–1308. DOI: 10.1109/TVCG.2017.2731949 3.
- [YKJM12] YUKSEL, CEM, KALDOR, JONATHAN M., JAMES, DOUG L., and MARSCHNER, STEVE. “Stitch Meshes for Modeling Knitted Clothing with Yarn-Level Detail”. *ACM Trans. Graph.* 31.4 (July 2012). ISSN: 0730-0301. DOI: 10.1145/2185520.2185533 2, 3.
- [YSL*24] YUAN, CHUN, SHI, HAORYANG, LAN, LEI, et al. “Volumetric Homogenization for Knitwear Simulation”. *ACM Trans. Graph.* 43.6 (Nov. 2024). ISSN: 0730-0301. DOI: 10.1145/3687911 3.
- [ZIA*23] ZHU, JUNQIU, JARABO, ADRIAN, ALIAGA, CARLOS, et al. “A Realistic Surface-Based Cloth Rendering Model”. *ACM SIGGRAPH 2023 Conference Proceedings*. SIGGRAPH ’23. Los Angeles, CA, USA: Association for Computing Machinery, 2023. ISBN: 9798400701597. DOI: 10.1145/3588432.3591554 2, 3.
- [ZJMB11] ZHAO, SHUANG, JAKOB, WENZEL, MARSCHNER, STEVE, and BALA, KAVITA. “Building Volumetric Appearance Models of Fabric Using Micro CT Imaging”. *ACM Trans. Graph.* 30.4 (July 2011). ISSN: 0730-0301. DOI: 10.1145/2010324.1964939 2, 3.
- [ZLB16] ZHAO, SHUANG, LUAN, FUJUN, and BALA, KAVITA. “Fitting procedural yarn models for realistic cloth rendering”. *ACM Trans. Graph.* 35.4 (July 2016). ISSN: 0730-0301. DOI: 10.1145/2897824.2925932 2, 3, 10.